🐠 Leading Through Code — The Game-Build SOP Book

Every video from the camp, as friendly step-by-step recipes. Open one at a time, follow the steps, tick it off. The ▶ pills jump to that exact moment in the video.

Parts / node names · Inspector · Files & folders · Menus & Play · ▶ video moments

1 DAY 1 Create a Simple Comic Book Template in Canva

This SOP explains how to find, open, and customize a comic book template in Canva. The goal is to help a team member quickly build a simple digital comic book or presentation using Canva’s templates and elements.

🎥 Watch the whole video

1 Open Canva and go to Templates ▶ 0:10

- Go to Canva.com and wait for the homepage to load.
- Click Templates from the main menu.
- This is the starting point for finding a comic book layout that fits your project.

2 Search for a comic book template ▶ 0:20

- In the Templates search bar, type comic book template.
- Review the results to see the available layout options.
- Compare the styles to choose one that matches the look and feel you want for your comic.

3 Choose a template that fits your comic style ▶ 0:28

- Browse the template options and identify one that best supports your story or presentation.
- Consider whether the layout is suitable for panels, text placement, and visual flow.
- Select a template that can be customized with your own content and design elements.

4 Open the selected template and review customization options

▶ 0:43

- Click the template to open it in Canva.
- Review the layout and determine where you can add or replace content.
- Use the template as a base for building out your comic book pages.

5 Add elements to build out the comic

▶ 1:24

- Use Canva's side panel to access elements, including:
  - Graphics
  - Pictures
  - Other visual assets
- Add relevant visuals to support the story and enhance the comic's appearance.
- If needed, choose a free template or element set to avoid paid content.

6 Use Canva as a tool to complete the comic book

▶ 1:39

- Continue adding and arranging elements until the comic book is complete.
- Use the template and available assets to create a polished digital comic or presentation.
- Save and share the finished design according to your team's workflow.

⚠ Watch out

- Free vs. paid content: Some templates and elements may require payment. Confirm whether the selected template is free before using it.
- Template fit: Not every search result will be a true comic book layout; some may be presentation-style designs.
- Copyright/usage: Only use images, graphics, and assets that are approved for your project or properly licensed.
- Consistency: Keep the visual style consistent across pages so the comic looks cohesive.

⚡ Go faster

- Start with a free template to save time and avoid licensing issues.
- Use the search bar with specific terms like comic book template to narrow results quickly.
- Pick a layout that already matches your intended panel structure to reduce editing time.
- Reuse similar elements across pages to maintain a consistent style and speed up design work.

- Build the comic in stages: choose template first, then add visuals, then refine text and layout.

2 DAY 1 Create a Basic Video Game Mood Board in Canva

Use Canva Whiteboards to build a simple mood board that defines the game's story, setting, characters, and core gameplay mechanics. This SOP helps a team member quickly organize visual references and ideas before moving into game design or production.

Watch the whole video

1 Open Canva Whiteboards and start a new mood board ▶ 0:43

- Open a new browser tab and go to Canva.com.
- From the Canva home screen, select Whiteboards.
- Create a new whiteboard to use as the mood board workspace.
- Treat this whiteboard as the central place to collect all visual and written ideas for the game.

2 Add a title for the mood board ▶ 1:17

- Click Text in Canva and add a heading at the top of the board.
- Title the board something clear, such as My Video Game Mood Board.
- Zoom out if needed so the full board layout is easy to view.
- Use the title to keep the board organized and easy to identify later.

3 Define the main idea of the game ▶ 1:47

- Add a new heading labeled Main Idea.
- Insert a text box underneath and write a short story premise.
- Include the core goal of the game, such as:
  - who the main player is,
  - what problem they must solve,
  - what they are trying to restore or protect,
  - what the main threat or enemy is.
- Keep the description brief but specific enough to guide the rest of the design.

4 Build the setting with visual references

▶ 3:07

- Add another topic area for Setting.
- Go to Elements in Canva and search for visuals that match the desired world.
- Add images or graphics that reflect the environment, such as:
  - a pink or purple sky,
  - forest trees,
  - background scenery that matches the game's tone.
- Arrange the visuals so they communicate the overall atmosphere of the game world.

5 Add character inspiration

▶ 4:29

- Create a new heading for Characters.
- Decide what the main character and enemies should roughly look like.
- Use Elements to search for character ideas, such as:
  - a pink animal hero,
  - a pig-like character,
  - a slime monster or other enemy type.
- Place the visuals on the board to help define the style and personality of the characters.

6 List core gameplay mechanics

▶ 5:39

- Add a final heading for Gameplay Mechanics.
- Write down the main actions the player should be able to perform.
- Include basic platformer mechanics such as:
  - running,
  - jumping,
  - climbing platforms,
  - collecting coins.
- Use this section to clarify how the game should feel and what the player will do moment to moment.

7 Review the completed mood board

▶ 6:24

- Review the full board to confirm it includes all major design areas:
  - main idea,
  - setting,
  - characters,
  - gameplay mechanics.
- Check that the visuals and text work together to communicate the intended game style.
- Make sure the board gives a clear overall impression of the game before moving to the next design step.

⚠️ **Watch out**

- Do not try to make the mood board perfect; it is meant to be a rough visual direction, not a final design.
- Avoid adding too many unrelated images, since that can make the board confusing.
- Keep the story, setting, and mechanics aligned so the board reflects one consistent game concept.
- If a visual does not match the intended tone, replace it rather than forcing it into the layout.

⚡ **Go faster**

- Start with broad ideas first, then refine details later.
- Use Canva search terms that are simple and descriptive, such as pink sky, forest, slime monster, or coins.
- Keep text short so the board stays easy to scan.
- Group related items together visually to make the mood board easier to understand at a glance.
- Use the whiteboard as a brainstorming tool before investing time in detailed game assets.

3 DAY 1 Open Godot Web Editor and Create a New 2D Project ▼

This SOP explains how to access the Godot Web Editor, create a new project, and switch the editor to 2D mode. It is intended to help a team member quickly set up a new Godot project and understand the basic editor layout.

📹 Watch the whole video

1 Access the Godot Web Editor ▶ 0:48

- Open a web browser.
- Search for Godot Web Editor.
- Select the Godot Engine Web Editor result.
- Confirm you are on the web-based editor page before proceeding.

2 Launch the Editor ▶ 1:00

- On the Godot Web Editor page, click Start Godot Editor.
- Wait for the editor to load fully.
- Review the screen for any preloaded projects that may already be listed.

3 Create a New Project ▶ 1:14

- Click the plus / Create button to start a new project.
- Enter a clear project name, such as a working title for the game.
- Verify the default folder location is acceptable, since Godot will automatically create the project folder on your computer.
- Click Create to generate the project and open it in the editor.

4 Switch to 2D Mode ▶ 1:48

- After the project opens, note that Godot starts in 3D mode by default.
- Click the 2D tab to switch the workspace to 2D mode.
- Confirm the editor view changes to the 2D viewport before continuing.

5 Review the Editor Layout ▶ 2:05

- Identify the main areas of the editor:
  - Viewport in the center for viewing and editing the scene.
  - Scene panel on the left for managing scenes and nodes.
  - **Inspector** and Signals on the right for editing object properties and connections.
- Use this layout as the starting point for future project setup tasks.

6 Test the Project When Needed ▶ 2:20

- Use the **Play button** at the top of the editor to run a test of the project.
- Check the game preview to confirm the project opens and runs as expected.
- Use testing early to catch setup issues before building additional content.

⚠️ **Watch out**

- The web editor is free and easier to use, but it does not include all features available in the desktop version.
- Make sure you are using the web editor if your workflow depends on the simplified setup described here.
- Godot opens in 3D mode by default, so remember to switch to 2D if you are building a 2D game.
- The project folder is created automatically on your computer when you create the project; confirm the name and location before clicking Create.

⚡ **Go faster**

- Use a simple, descriptive project name so it is easy to identify later.
- Keep the default folder location unless your team has a specific file organization standard.
- Switch to 2D mode immediately after creating the project to avoid working in the wrong workspace.
- Familiarize yourself with the viewport, scene panel, inspector, and signals early to speed up future tasks.
- Use the **Play button** frequently during setup to verify that the project is functioning correctly.

4 DAY 1 Import Assets into Godot and Organize Them in an Assets Folder ▼

This SOP explains how to source, download, extract, and import game assets into a Godot project. It also covers how to organize the imported files into a dedicated assets folder so the project stays clean and easy to manage.

🎥 Watch the whole video

1 Choose or create the assets you need ▶ 0:21

- Review your mood board and decide what assets are required for your game.
- Choose one of the following asset sources:
  - Create your own sprites in a tool like Piskel.
  - Download free assets from itch.io.
  - Use a [precompiled asset library](https://drive.google.com/drive/folders/1z-KcEHrIws3qpExZPyMkwqEfgIXIH-7g?usp=drive_link) provided for the project.
- Make sure the assets match the look and feel planned in your mood board.

2 Download the asset package

▶ 1:08

- If using a shared asset library, open the provided Google Drive link.
- Click the download arrow next to the asset folder or file.
- Save the download to your computer.
- If using itch.io, download the asset pack from the website in the same way.
- Expect the download to arrive as a zipped/compressed file.

3 Extract the downloaded files

▶ 3:13

- Open your Downloads folder.
- Locate the downloaded compressed file.
- Right-click or open the file and select Extract All.
- Confirm the extraction so the contents are unzipped into a normal folder.
- Verify that the extracted folder contains the asset files you need.

4 Open the Godot project and create an assets folder

▶ 4:02

- Open your Godot project in the editor.
- In the File System panel, create a new folder for organization.
- Name the folder Assets.
- Use this folder as the central location for all imported game assets.

5 Import the extracted assets into Godot

▶ 4:21

- Return to your Downloads folder.
- Find the extracted asset folder.
- Click and drag the folder into the Godot File System panel.
- Wait for Godot to copy the files into the project.
- Confirm that the assets now appear inside the project file structure.

6 Move the imported files into the Assets folder

▶ 4:36

- To keep the project organized, select the imported files.
- Click the first item, scroll to the bottom, hold Shift, and click the last item to select the full range.
- Drag the selected files into the **Assets folder**.
- When prompted, confirm the move.
- Verify that all imported assets are now stored inside the **Assets folder** and ready for use.

⚠ Watch out

- Do not skip extraction: zipped files must be uncompressed before importing into Godot.
- Keep assets organized: place all imported files in a dedicated Assets folder to avoid confusion later.
- Check usage rights: only use assets that are free to use or properly licensed for your project.
- Confirm file integrity: after importing, make sure the files appear correctly in Godot and were not missed during the drag-and-drop process.

⚡ Go faster

- Use a precompiled asset library when you need to move quickly.
- Keep your mood board nearby so you can choose assets that match your intended style.
- If you plan to create custom sprites, design them before importing so you only bring in final versions.
- Use multi-select with Shift to move many files at once instead of organizing them one by one.
- Standardize folder naming early so every team member uses the same structure.

5 DAY 1 Create a Player Character Scene in Godot 2D

This SOP explains how to set up a main game level, create a player character as its own scene, add animations and collision, and save everything in an organized project structure. Follow these steps to build a reusable player object that can later be scripted for movement and gameplay interactions.

🎥 Watch the whole video

1 Create the main game level root node ▶ 1:42

- In Godot, create the primary 2D scene that will act as the container for the entire game.
- Click 2D Scene to create a `Node2D`.
- Rename the node to GameLevel.
- Treat this as the root/parent node for the game, where all other objects will be added as children.

2 Organize the project with folders

▶ 2:34

- Create a clean folder structure before saving scenes.
- Confirm the project already has an **Assets folder** for art and other resources.
- Create a **Scenes folder** for all Godot scene files.
- Create a **Scripts folder** for code files used to control objects.
- Keep this structure consistent so assets, scenes, and scripts are easy to find.

3 Save the main game level scene

▶ 3:30

- Save the newly created root scene immediately.
- Go to Scene > Save Scene As.
- Save the file inside the **Scenes folder**.
- Use the name GameLevel.
- This scene will serve as the main level for the game.

4 Create the player scene using `CharacterBody2D`

▶ 4:20

- Create a new scene for the player character.
- Click the Add a New Scene button.
- Choose Other Node and search for `CharacterBody2D`.
- Create the node and rename it to Player.
- Use this node type because it is designed for a controllable 2D character.

5 Add child nodes for animation and collision

▶ 5:22

- With Player selected, add child nodes so the character can both animate and interact with the world.
- Add an `AnimatedSprite2D` node for the character's visual animation.
- Add a `CollisionShape2D` node for physical interaction and collisions.
- Make sure both nodes appear indented under Player, confirming they are children of the player node.

6 Configure the player's idle animation

▶ 6:46

- Select `AnimatedSprite2D` and locate Sprite Frames in the **Inspector**.
- Create a new `SpriteFrames` resource.
- Open the animation frame editor and choose Add Frames from Sprite Sheet.
- Browse to the **Assets folder** and select the paint monster idle sprite sheet.
- Set the sheet slicing correctly so each frame is isolated:
  - Horizontal frames: 4
  - Vertical frames: 1
- Select all four frames and add them to the animation.

7 Improve sprite clarity and finalize idle playback

▶ 10:10

- If the sprite appears blurry, adjust the project texture filtering.
- Go to Project > **Project Settings**.
- Under Rendering > Textures, change the filter from Linear to Nearest.
- Return to the animation editor and verify the sprite looks sharper.
- Set the idle animation frame rate to about 10 for smoother motion.
- Rename the animation from Default to Idle.
- Enable looping so the idle animation repeats continuously.
- Turn on Autoplay on Load so the idle animation starts automatically when the game runs.

8 Add the jump animation

▶ 11:54

- Add a second animation for jumping.
- In the animation panel, click Add Animation and name it Jump.
- Use Add Frames from Sprite Sheet again.
- Select the jump sprite sheet from the paint monster assets.
- Slice the sheet correctly:
  - Horizontal frames: 8
  - Vertical frames: 1
- Select all frames and add them.
- Set the frame rate to around 10 and preview the animation to confirm it plays correctly.

9 Add the run animation

▶ 13:17

- Add a third animation for running.
- Click Add Animation and name it Run.
- Load the run sprite sheet from the paint monster assets.
- Slice the sheet correctly:
  - Horizontal frames: 6
  - Vertical frames: 1
- Select all six frames and add them.
- Set the frame rate to around 10 and preview the animation to confirm it loops and plays smoothly.

10 Add and size the collision shape

▶ 14:01

- Select the `CollisionShape2D` child node.
- In the **Inspector**, choose a CircleShape2D for the collision shape.
- Resize and position the circle so it fits around the player character.
- Make the collision shape slightly smaller if needed so it aligns well with the sprite.
- Confirm the player now has both visual animation and collision behavior.

11 Save the player scene

▶ 14:56

- Save the player as its own reusable scene.
- Go to Scene > Save Scene As.
- Save it inside the **Scenes folder**.
- Name the scene Player.
- This creates a standalone player scene that can later be added to the main game level.

⚠ Watch out

- Always save new scenes immediately to avoid losing work.
- Make sure nodes are selected before adding child nodes; otherwise, the new node may not attach to the player.
- Use the correct sprite sheet dimensions when slicing frames; incorrect horizontal/vertical values will break the animation layout.
- If the sprite looks blurry, change texture filtering to Nearest before judging the final appearance.
- Keep the collision shape aligned with the sprite so gameplay interactions feel accurate.

- Do not forget to set the idle animation as autoplay if you want the character to animate automatically on game start.

⚡ Go faster

- Create the Scenes and Scripts folders first so files stay organized from the beginning.
- Reuse the same workflow for each animation: add animation → load sprite sheet → set frame grid → add frames → preview.
- Use Idle, Jump, and Run as a simple starter animation set before adding more complex actions.
- Keep the idle animation looping and set it as the default to reduce setup later.
- Save the GameLevel and Player scenes separately so the player can be reused in other levels if needed.

6 DAY 1 Add a Player Scene to the Game Level and Set Up a Basic Camera for Playtesting

This SOP explains how to place a saved player scene into the main game level, duplicate or remove instances as needed, and add a `Camera2D` so the player can be viewed during playtesting. It also covers how to run a quick test to confirm the player appears on screen.

🎥 Watch the whole video

1 Drag the saved player scene into the game level ▶ 0:32

- Open the main Game Level scene.
- Locate the saved Player scene in the Scenes panel.
- Drag the Player scene directly into the workspace.
- Confirm the player instance appears in the level.
- Treat the saved player scene like a reusable blueprint that can be placed multiple times if needed.

2 Add or remove player instances as needed ▶ 0:43

- If you need multiple copies of the player scene, drag additional instances into the level.
- Verify each copy appears in the workspace.
- Remove any extra player instances that are not needed.
- Keep only the intended player object(s) in the scene to avoid confusion during testing.

3 Prepare the scene for playtesting by adding a `Camera2D` ▶ 1:08

- Select the Game Level node.
- Search for and create a `Camera2D` node.
- Add the camera to the scene so the player can be viewed during play.
- Zoom out slightly if needed to locate the camera outline in the workspace.

4 Position and adjust the camera around the player ▶ 1:26

- Drag the `Camera2D` so it is centered on the player.
- Check the camera size relative to the player.
- Adjust the camera zoom to frame the character more appropriately.
- Try a zoom value such as 3, then reduce it if the view is too tight; for example, test 2 for a closer fit around the character.

5 Run a playtest to confirm the player appears on screen ▶ 1:44

- Click the **Play button** in the top-right corner.
- Confirm the player is visible in the game window.
- Use the playtest to verify the camera is showing the correct area of the scene.
- Observe the character on screen to ensure the scene is loading correctly.

6 Verify current movement behavior during testing ▶ 2:03

- Press movement keys such as W, A, S, D or the arrow keys.
- Confirm that the player does not move yet if movement has not been programmed.
- Use this test only to verify that the player is visible and ready for future movement logic.
- Note that the character can be displayed and animated even before movement controls are implemented.

⚠ Watch out

- Do not assume the player will move during this step; movement code has not been added yet.
- Avoid leaving extra player instances in the scene unless multiple players are intentionally required.
- Make sure the camera is centered properly; otherwise, the player may appear off-screen during playtesting.
- If the camera view looks too large or too small, adjust zoom before finalizing the test.

Go faster

- Use the saved player scene as a reusable asset to quickly place consistent player objects in future levels.
- Center the `Camera2D` on the player immediately after adding it to reduce setup time.
- Test zoom values quickly to find the best framing for the character.
- Run a short playtest after each major change to confirm the player remains visible and the scene is configured correctly.

7 DAY 2 Add a Player Script, Create a Temporary Ground Boundary, and Test Basic Movement

This SOP explains how to attach a built-in script to the player node, save the scene, add a temporary ground using a Static Body 2D with a World Boundary Shape, and test the player's basic left/right movement in the game. It is intended to help a team member verify that the player can move without falling before further gameplay mechanics are added.

Watch the whole video

1 Close the running game and return to the editor ▶ 0:17

- If the game is currently running, click the X button next to the game window at the top.
- Click Editor to return to the project editor.
- Confirm you are back in the editor before making changes.

2 Select the player node and add a script ▶ 0:37

- Open the Player scene.
- Make sure the Player node is highlighted so the script is attached to the correct node.
- Click the script/add script button (the parchment icon with a green plus sign).
- In the script creation window, choose where to save the script by clicking the folder button.
- Select the scripts folder, then click Open.
- Use the built-in template rather than an empty script.
- Click Create to attach the script to the player.

3 Confirm the script is attached and save the scene

▶ 1:49

- Verify the player node now shows the script icon, indicating a script is attached.
- Review the script briefly if needed, but do not change the code yet.
- Save the scene by going to Scene > Save Scene.
- Confirm the Player scene is saved before testing.

4 Test the player in the game and observe gravity

▶ 2:30

- Return to the game level view by clicking 2D at the top.
- Press the **Play button** to run the game.
- Observe that the player now falls immediately when the game starts.
- Note that this happens because the script adds gravity to the player.

5 Add a temporary ground using a Static Body 2D

▶ 3:53

- Select the Game Level node.
- Click the plus button to add a new node.
- Search for and add Static Body 2D.
- With Static Body 2D selected, click the plus button again.
- Add a Collision Shape 2D child node.
- In the inspector on the right, set the collision shape to World Boundary Shape.
- Position the boundary horizontally underneath the player so it acts as a temporary ground.

6 Save all scenes and retest movement

▶ 5:42

- Save all scenes to preserve the new boundary setup.
- Click the **Play button** again to retest the game.
- Confirm the player no longer falls through the level.
- Press the left and right keys to verify the player can move horizontally.
- Observe that other controls do not work yet because they have not been programmed.

7 Review current limitations before the next update

▶ 6:26

- Note that the camera is static and does not follow the player.
- Observe that the player may move out of view as they travel.
- Notice that the player movement speed is currently very fast.
- Prepare to adjust movement speed and camera behavior in the next development step.

⚠️ **Watch out**

- Do not attach the script to a child node by mistake; always select the main Player node first.
- Save the script in the correct scripts folder to keep the project organized.
- The World Boundary Shape is only a temporary ground solution for testing movement.
- If the player still falls, confirm the collision shape is correctly added and positioned beneath the player.
- The script introduces gravity, so falling is expected until a ground or boundary is added.

⚡ **Go faster**

- Use the built-in script template to speed up setup and avoid starting from scratch.
- Save the scene immediately after attaching the script to prevent losing changes.
- Add the temporary ground before testing movement so you can focus on controls instead of falling physics.
- Keep the boundary simple and horizontal for quick validation of player movement.
- Test after each major change so issues are easier to isolate.

8 DAY 2 Adjust Player Movement Speed by Editing a Constant in the Player Script ▼

This SOP explains how to locate the player script, identify the speed constant, adjust the movement speed value, save the changes, and test the game to confirm the player moves at the desired pace. It is intended to help a team member quickly tune character movement without changing other gameplay systems.

🎥 Watch the whole video

1 Open the Player Script ▶ 0:24

- Select the player character in the scene or hierarchy.
- Open the attached script using the script button next to the player.
- Confirm the player script is visible before making any edits.

2 Locate the Speed Constant

▶ 0:35

- Find the line that defines the movement speed, such as const speed = 300.
- Understand that const means constant, which is a value intended to stay the same unless you manually edit it.
- Recognize that this number controls how fast the player moves.

3 Reduce the Movement Speed Value

▶ 1:54

- Edit the speed value to a lower number to slow the character down.
- Example change: reduce 300 to 150.
- Choose a value that feels appropriate for your game's movement style.

4 Save the Script Changes

▶ 2:10

- Save the script after updating the speed value.
- Make sure the change is stored before testing.
- If your workflow requires it, also save the scene/project to preserve the update.

5 Test the Game and Verify the Result

▶ 2:17

- **Run the game** and move the player left or right.
- Confirm the character now moves slower than before.
- If needed, return to the script and fine-tune the speed value until the movement feels right.

⚠️ Watch out

- Do not confuse constants with variables: constants are meant to remain fixed unless you intentionally edit them.
- Changing the speed value affects movement immediately, so test after every adjustment.
- The character's animation may not yet match the movement speed; run animations can be added later.
- Avoid making the speed too low or too high, as it can make the game feel unresponsive or overly fast.

⚡ Go faster

- Make small speed adjustments in increments so it is easier to find the best feel.
- Keep a few test values in mind (for example, 300, 200, 150) to compare movement quickly.
- Save and test after each change instead of making multiple edits at once.

- Once the desired speed is found, document the chosen value for future reference.

9 DAY 2 Create a 2D Platformer Foreground, Collision, and Camera Follow Setup in Godot

This SOP explains how to build a 2D platformer scene using `TileMap` layers, paint platforms and decorations, assign collision only to playable surfaces, and attach the camera to the player for smooth following. It is designed so a team member can recreate the setup consistently and test it successfully.

Watch the whole video

1 Remove the temporary ground placeholder ▶ 0:12

- In the scene tree, locate the temporary ground/static body created earlier.
- Right-click the static body and delete it.
- Confirm the placeholder ground is removed before building the final level layout.

2 Add a `TileMap` layer for the foreground ▶ 1:47

- Select Game Level so the new node becomes its child.
- Click the + button and add a TileMapLayer node.
- Rename the new layer to Foreground.
- Use this layer for platforms and interactive level pieces.

3 Create and configure a new TileSet ▶ 2:38

- In the **Inspector**, find the Tile Set field for the foreground layer.
- Click New TileSet.
- Set the tile size to 16x16 unless your art uses a different matching size.
- Make sure the tile size matches the source art dimensions exactly.

4 Import the tile atlas into the TileSet ▶ 3:41

- In the TileSet editor, ensure the Tileset source is selected.
- Drag world tileset.png into the tile source area.
- When prompted, choose Yes to automatically create the atlas tiles.
- Verify the tiles appear in the TileSet grid.

5 Clean up tile grouping for better painting

▶ 5:04

- Review the auto-sliced tiles for any pieces that should stay together.
- Use the Erase tool to remove unwanted tile splits.
- Hold Shift and drag to regroup tile parts when needed.
- Keep tree tops or other decorative pieces grouped if you want them to behave as a single visual element.

6 Paint the foreground platforms

▶ 6:00

- Switch to Tile Map mode to begin painting.
- Select a platform tile from the TileSet.
- Click on the grid to place tiles and right-click to remove them.
- Build a simple platform layout with gaps for jumping and movement.
- Use click-and-drag to place multiple tiles quickly.

7 Add decorative foreground elements

▶ 8:41

- After the platform layout is complete, add decorative items such as trees, mushrooms, or fruit.
- Use the separated tile parts to vary object height and shape.
- Place decorations where they enhance the scene without blocking gameplay.
- Keep decorative placement visually interesting but simple enough to read clearly.

8 Add a physics layer for collision tiles

▶ 10:32

- In the **Inspector**, open Physics Layer for the TileSet.
- Click Add Element to create a collision layer.
- Return to the TileSet editor and switch to the Paint tool.
- Select the physics layer so you can assign collision only to the tiles that should be solid.

9 Paint collision only on playable surfaces

▶ 11:03

- Paint the physics layer onto platform tiles that the player should stand on.
- Do not paint collision onto decorative objects like trees or mushrooms.
- If you accidentally paint the wrong tile, use the three-dot menu and choose Clear.
- Use Reset to Default Tile Shape if you need to restore or adjust the collision shape.

10 Test the level and verify player collision

▶ 12:41

- Return to the main Game Level scene.
- Save all scenes.
- **Run the game** to test the level.
- Confirm the player stands on the platforms instead of falling through them.
- If the player falls through, revisit the collision painting step.

11 Raise the player above foreground objects using Z-index

▶ 13:37

- Select the Player node in the scene tree.
- In the **Inspector**, find the Z Index setting under ordering.
- Set the Z Index to a higher value, such as 5.
- Re-run the game to confirm the player renders in front of foreground decorations.

12 Attach the camera to the player

▶ 16:07

- In the scene tree, move the Camera node underneath the Player node.
- This makes the camera a child of the player instead of the game level.
- **Run the game** and confirm the camera follows the player's movement.
- Enable Position Smoothing for smoother camera motion.

13 Finalize the foreground before adding the background

▶ 17:17

- Review the foreground layout to ensure platforms, collision, and camera behavior are working.
- Make any final adjustments to platform spacing or decoration placement.
- Once the foreground is complete, proceed to paint the sky/background in a separate step or video.
- Keep moving platforms and other advanced mechanics for later implementation.

⚠ Watch out

- Do not assign collision to decorative tiles unless they are meant to be solid.
- Make sure the tile size in the TileSet matches the source art dimensions.
- If the player appears behind foreground objects, check the player's Z Index.
- If the camera does not follow the player, confirm it is parented under the Player node.
- Save the scene before running tests to avoid losing changes.

⚡ Go faster

- Build the platform layout first, then add decorations afterward.
- Use click-and-drag painting to place repeated tiles faster.
- Reuse separated tile parts to create trees of different heights without needing new art.
- Keep collision shapes as simple as possible for easier debugging.
- Test frequently after each major change to catch issues early.

10 DAY 2 Create and Organize a Separate Background Layer for a Game Scene

This SOP explains how to paint a game background on its own tile map layer, set draw order correctly, and group scene layers into a single settings node. Following these steps keeps the background editable without affecting gameplay platforms.

🎥 Watch the whole video

1 Create a separate background tile map layer ▶ 0:21

- In the main scene node, select the existing Game Level tile map layer.
- Click the plus (+) button to add a new Tile Map Layer.
- Click Create to generate the new layer.
- Rename the new layer to Background.
- Keep the background separate from the foreground so you can paint freely without affecting gameplay platforms.

2 Set the draw order so the background renders behind gameplay ▶ 1:08

- Decide which layer should render first.
- Either move Background above Foreground in the scene tree, or adjust the Z-index directly.
- Recommended approach from the transcript:
  - Go to Foreground.
  - Open Ordering.
  - Set the Z-index to 2.
- Leave the background at the default lower Z-index so it renders behind the foreground and player.

3 Add the same tileset to the background layer

▶ 1:39

- Select the Background layer.
- Click the tile button next to the layer.
- Add the same tileset used for the foreground.
- Use background-friendly tiles already included in that tileset.
- Do not add a physics layer to the background, since it is not meant for player interaction.

4 Paint the background using the chosen color palette

▶ 2:19

- Refer to the mood board or visual direction before painting.
- Choose background colors that match the intended theme (for example, pinks and purples).
- Open the Tile Map palette and select a background tile color.
- Start painting the background area across the scene.
- Work in broad strokes first, then fill in gaps with additional tiles for a layered look.

5 Verify the background is rendering behind the foreground

▶ 3:01

- Check the scene after painting to confirm the background appears behind the foreground.
- Remember that a higher Z-index means higher draw priority.
- Confirm the draw order is correct:
  - Player at the highest priority (example: Z-index 5)
  - Foreground above background (example: Z-index 2)
  - Background at the default lowest priority (example: Z-index 0)
- Continue painting until the background fills the visible scene area.

6 Refine the background coverage and fill visible gaps

▶ 4:07

- Zoom out to inspect the full camera view.
- Add more tiles to areas that look empty or unfinished.
- Mix tile colors to create a more natural or decorative background.
- Continue filling the scene until the background looks complete enough for the game’s style.
- If needed, use additional tile variations to create texture and depth.

7 Test the scene and correct any missing background areas

▶ 5:31

- Run a test of the scene to confirm the background displays correctly.
- If the background does not appear, reload the game or scene and test again.
- Inspect the visible camera area for any unpainted gray sections.
- Paint any missing areas so the full camera view is covered.
- If a section looks wrong, remove tiles by right-clicking and repainting as needed.

8 Separate and organize scene layers under a settings node

▶ 7:35

- In the main scene tree, select Game Level.
- Click the plus (+) button and add a regular Node.
- Click Create.
- Rename the node to Settings.
- Drag both Foreground and Background into the Settings node.
- Confirm the draw order still works correctly after grouping the layers.

⚠️ **Watch out**

- Do not add physics to the background layer unless the background must interact with gameplay.
- Be careful not to paint over gameplay platforms; keeping background and foreground separate prevents accidental edits.
- If the background is not visible, check the layer order and reload the scene if necessary.
- Make sure the background covers the full camera view, including the bottom edge, to avoid visible gray areas.

⚡ **Go faster**

- Use a separate tile map layer for the background from the start to simplify editing later.
- Keep the foreground and background organized under a single parent node for cleaner scene management.
- Use broad painting first, then refine details only where needed.
- Reuse the same tileset for both foreground and background if it already contains suitable background tiles.
- Adjust Z-index instead of manually rearranging visuals when you only need to control draw order.

11 DAY 2 Create and Organize Platform Objects in a 2D Game Scene

This SOP explains how to create a reusable platform scene, configure its sprite and collision, place it into the main game level, and organize multiple platforms for easier scene management. It is intended to help a team member add static and future moving platforms consistently and correctly.

Watch the whole video

1 Create a dedicated platform scene ▶ 0:29

- In the editor, click the Add Child Node (+) button.
- Search for and select AnimatableBody2D.
- Rename the node to Platform.
- Use a separate scene for the platform so it can be reused throughout the game.
- Keep the platform as its own object rather than building it directly into the main level.

2 Add a visible sprite to the platform ▶ 0:57

- Add a `Sprite2D` node under the Platform.
- Use `Sprite2D` instead of `AnimatedSprite2D` if the platform does not need animation yet.
- In the Texture field, drag in the platform asset from the project files.
- Confirm the texture loads correctly before moving on.

3 Crop the sprite to a single platform image ▶ 1:26

- If the texture contains multiple platform variations, enable Region.
- Click Edit Region to open the sprite region editor.
- Change the snap mode to Pixel Snap for precise selection.
- Select only the platform image you want to use.
- Close the editor and verify the sprite now displays as a single platform.

4 Add collision to the platform ▶ 2:04

- Select the Platform node and add a `CollisionShape2D` node.
- Choose a `RectangleShape2D` since the platform is rectangular.
- Resize and position the collision shape so it fits the visible platform sprite.
- Make sure the collision area matches the platform edges closely for accurate gameplay.

5 Save the platform as a reusable scene

▶ 2:32

- Save the platform as its own scene using Scene > Save Scene As.
- Store it in the project's **Scenes folder**.
- Confirm the platform scene appears in the file system for reuse.
- This allows the platform to be dragged into the main game level multiple times.

6 Place platforms into the main game level

▶ 2:44

- Open the main game level scene.
- Drag and drop the saved platform scene into the level.
- Duplicate or add as many platforms as needed to build the traversal path.
- Position platforms where the player should be able to jump and land.

7 Fix platform visibility using z-index ordering

▶ 3:08

- If a platform is hidden behind the background, select the platform instance.
- Go to Ordering and adjust the z-index.
- Increase the platform's z-index value so it renders in front of the background.
- Verify the platform is visible in the game view after the change.

8 Configure one-way collisions for jump-through behavior

▶ 4:14

- Open the platform scene settings.
- Enable One-Way Collisions.
- This allows the player to jump up through the platform from below and land on top.
- Use this setting for standard floating platforms that should be passable from underneath.

9 Group all platform instances under a single parent node

▶ 5:03

- In the main game level, add a new Node and rename it to Platforms.
- Drag all platform instances into the new Platforms node.
- Keep static and moving platforms organized in one place.
- Use this structure to simplify scene management and future edits.

⚠ Watch out

- Ensure the platform sprite is cropped correctly; leaving the full texture visible can cause visual errors.
- Match the collision shape to the sprite size to avoid unfair or broken player interactions.

- If a platform does not appear, check whether it is behind the background and adjust the z-index.
- Enable one-way collisions only when the platform should be jump-through from below; do not use it for solid walls or blocking surfaces.
- Save the platform as a separate scene before placing multiple copies in the level to avoid repetitive manual setup.

⚡ Go faster

- Reuse the saved platform scene instead of rebuilding each platform from scratch.
- Organize all platform instances under a single Platforms parent node to keep the scene tree clean.
- Use pixel snap when selecting sprite regions for more accurate cropping.
- Set up one platform first, then duplicate it to speed up level building.
- Adjust z-index early if your level uses layered backgrounds to prevent visibility issues later.

12 DAY 2 Create a Ping-Pong Moving Platform Using an Animation Player ▼

This SOP explains how to add an Animation Player to a platform, create a keyframed movement animation, and configure it to loop back and forth. The result is a reusable moving platform that can be applied to one object without affecting other copied platforms.

🎥 Watch the whole video

1 Add an Animation Player to the target platform ▶ 0:24

- Select Platform 3 in the scene.
- Click the plus (+) button to add a new node/component.
- Search for Animation Player and create it.
- This node will control the platform’s movement animation.

2 Create a new animation track ▶ 0:41

- Open the Animation panel at the bottom of the editor.
- Click Add New Animation.
- Name the animation move.
- This animation will contain the platform’s movement keyframes.

3 Set the first keyframe at the starting position

▶ 0:51

- Use the timeline/keyframe editor to define movement over time.
- Return to Platform 3 and locate its Transform properties.
- Focus on the position property, since the goal is to move the platform horizontally.
- At time 0:00, click the keyframe button next to the position value to record the starting location.

4 Move the platform to the end position and add the second keyframe

▶ 1:49

- Move forward in the timeline to about 1 second.
- Select the platform and reposition it to the desired end point to the right.
- Hold Shift while moving to keep the motion aligned and prevent vertical drift.
- Once the platform is in the correct location, add another keyframe to capture the end position.

5 Test the animation and adjust the duration if needed

▶ 2:27

- Press Play to preview the animation.
- If the platform moves too quickly, extend the animation length.
- Move the second keyframe farther out on the timeline, such as to the 3-second mark.
- Play the animation again to confirm the movement is slower and smoother.

6 Enable looping and switch to back-and-forth playback

▶ 3:07

- Locate the Animation Looping option on the right side.
- Turn looping on to repeat the animation.
- If the animation restarts instead of reversing, change the loop mode.
- Select the back-and-forth / ping-pong mode so the platform travels right, then returns left automatically.

7 Verify the animation is isolated to the correct platform

▶ 3:40

- Confirm that only Platform 3 has the animation attached.
- Check that the other platforms remain static and do not move.
- This ensures the animation is applied only to the intended object.

8 Save the project

▶ 4:10

- Save the game/project after confirming the animation works correctly.
- Saving preserves the animation setup and prevents loss of work.

⚠ Watch out

- Do not forget to add the Animation Player to the correct platform before creating keyframes.
- Make sure keyframes are placed in the correct order on the timeline; otherwise, movement may behave unexpectedly.
- If the platform moves vertically when it should only move horizontally, hold Shift while repositioning it.
- Looping alone may restart the animation instead of reversing it; switch to back-and-forth/ping-pong mode if needed.
- Always save after testing to avoid losing the animation setup.

⚡ Go faster

- Use a short initial timeline span to quickly test movement, then extend it only if the animation is too fast.
- Reuse a basic scene and apply animations only to the specific copied object that needs movement.
- Keep the animation name simple and descriptive, such as move, for easier management.
- Use keyframes only on the properties you need, such as position, to keep the animation clean and easy to edit.

13 DAY 2 Configure Player Movement, Jump Input, and Platform Animation in the Game Project ▼

Set up player controls, update the player script to use custom input actions, test movement and jumping, and correct platform animation settings so the game can be playtested properly. This SOP also includes a quick validation pass to confirm movement, jump height, and platform motion are working as expected.

🎥 Watch the whole video

1 Open the player script and identify the input-related code

▶ 0:00

- Open the Player scene.
- Click the script icon beside the player node to open the player script.
- Locate the input-related lines in the script, especially the sections handling movement and jumping.
- Review the existing input names so they can be replaced with the custom actions you will create in the project settings.

2 Create custom input actions in Project Settings

▶ 1:08

- Go to Project > **Project Settings** > **Input Map**.
- Add a new action for moving left.
- Bind Left Arrow to the left action.
- Add A as an additional key for the same left action if using WASD controls.
- Add a new action for moving right.
- Bind Right Arrow to the right action.
- Add a new action named jump.
- Bind Spacebar to the jump action.
- Confirm the action names are exactly:
  - move_left
  - move_right
  - jump

3 Update the player script to match the new input names

▶ 3:08

- Return to the player script.
- Replace the old jump input name with the new action name jump.
- Replace the old left movement input name with move_left.
- Replace the old right movement input name with move_right.
- Save the script after updating the action names.
- Verify the script is now referencing the same names used in the **Input Map**.

4 Understand and verify the movement logic before testing

▶ 4:09

- Review how the movement code uses a direction value.
- Confirm the direction value behaves as follows:
  - -1 when moving left
  - 0 when idle
  - 1 when moving right
- Understand that movement speed is controlled by multiplying direction by speed.
- Confirm the code also includes gravity logic so the player falls back to the floor when not grounded.
- Check the smoothing logic that slows movement back toward zero when input stops.

5 Save and playtest the player controls

▶ 5:17

- Save the scene.
- **Run the game** using the play button.
- Test movement using:
  - A / Left Arrow for moving left
  - D / Right Arrow for moving right
  - Spacebar for jumping
- Confirm the player can move and jump successfully.
- Observe whether the jump height feels appropriate for the level.

6 Inspect and fix the moving platform animation settings

▶ 6:28

- Return to the game level.
- Select the third platform.
- Open the `AnimationPlayer` for that platform.
- Check the active animation and confirm it is set to the move animation, not reset.
- Enable Autoplay on Load so the animation starts automatically when the scene runs.
- Save the changes before testing again.

7 Adjust jump height and retest the game

▶ 7:26

- Open the player script again.
- Reduce the jump strength if the player is jumping too high.
- Update the jump value to a lower number, such as -300.
- Save the scene.
- **Run the game** again and retest movement, jumping, and platform motion.
- Confirm the jump now feels more controlled and the platform moves correctly.

8 Verify remaining animation behavior and note next steps

▶ 8:22

- Observe the player character during movement.
- Confirm whether the character still faces the same direction regardless of movement.
- Check whether idle, jump, and run animations are playing correctly.
- If animations are not switching yet, document that the next step is to update the code so the correct animation plays when moving left, jumping, or standing still.

⚠ Watch out

- Make sure the input action names in the script match the names in the Input Map exactly, including underscores and spelling.
- If the platform does not move, verify both the correct animation is selected and autoplay is enabled.
- If the jump feels too strong, lower the jump value and retest rather than making large changes all at once.
- Save the scene before each playtest to avoid testing outdated settings.
- If movement works but animations do not, the issue is likely in animation logic rather than input mapping.

⚡ Go faster

- Set up all input actions first before editing the script to avoid repeated back-and-forth.
- Test one change at a time so bugs are easier to identify.
- Keep a small list of expected controls nearby during playtesting:
- Left: A or Left Arrow
- Right: D or Right Arrow

14 DAY 2 Configure Player Animation Logic for Idle, Run, Jump, and Facing Direction

Set up the player script so the character's animations match movement input and facing direction. This SOP ensures the player idles when stationary, runs when moving, jumps when airborne, and flips left/right correctly.

🎥 Watch the whole video

1 Open the Player Script and Prepare to Control Animations ▶ 0:23

- Open the Player script.
- Identify the need to trigger different animations based on player input.
- Confirm the goal for each state:
  - Jump when the jump action is used.
  - Run left/right when movement keys are pressed.
  - Idle when no movement input is active.

2 Create a Reference to the `AnimatedSprite2D` Node ▶ 1:26

- In the Player scene, locate the `AnimatedSprite2D` child node.
- Drag `AnimatedSprite2D` into the script while holding Ctrl to auto-create a node reference variable.
- Rename the variable to something simple, such as AnimatedSprite.
- Understand that this variable stores the sprite node so the script can control animations directly.

3 Use the Sprite Reference to Play Animations ▶ 4:10

- Use the AnimatedSprite variable to access sprite behaviors.
- Apply the sprite's animation-playing function to switch between animation states.
- Keep the variable available for later logic that will determine which animation should play.

4 Configure Horizontal Flip for Left/Right Facing

▶ 4:57

- In the `AnimatedSprite2D` inspector, locate the Offset section.
- Find the Flip H option, which flips the sprite horizontally.
- Use Flip H to make the character face left when needed.
- Leave Flip V unused unless vertical flipping is required.
- Plan to control Flip H through code instead of manually toggling it in the inspector.

5 Read the Direction Input Value

▶ 6:11

- Locate the direction variable in the script.
- Use this value to determine which direction the player is facing.
- Interpret the direction values as follows:
  - -1 = facing left
  - 1 = facing right
  - 0 = no directional input / default facing state

⚠ Watch out

- Make sure indentation is correct for every if, elif, and else block.
- Use the exact syntax shown in the script, including colons, parentheses, and capitalization.
- Ensure the AnimatedSprite variable name matches everywhere it is referenced.
- If the code does not behave as expected, check for syntax errors before changing logic.
- Do not manually enable Flip H in the inspector if the script is meant to control it.

⚡ Go faster

- Use the drag-and-drop node reference method to avoid typing long node paths manually.
- Keep variable names short and consistent for easier reading and maintenance.
- Test one behavior at a time: facing direction, idle/run animation, then jump animation.
- Save the scene before running the game to verify changes quickly.
- Use the game test run to confirm the character matches movement input visually.

15 DAY 3 Create a Restart Zone to Reload the Game on Player Death or Boundary Exit

This SOP explains how to build a reusable restart zone in a game scene so the current level reloads when the player falls off the map or collides with a restart-triggering object. It also includes an optional slow-motion effect to make the restart sequence feel smoother and more polished.

Watch the whole video

1 Create a reusable Restart Zone scene ▶ 0:21

- Add a new scene using Area 2D as the root node.
- Rename the scene to Restart Zone.
- Use Area 2D because it detects when a body enters or exits the area.
- Do not add a collision shape yet if you want this scene to be reusable in multiple situations (for example, a bottom-of-level boundary or an enemy collision trigger).

2 Save the scene and attach a script ▶ 2:24

- Save the scene first before adding logic.
- Use Scene > Save Scene As and save it as restart zone in the scenes folder.
- Attach a new script to the Restart Zone scene.
- Save the script in the scripts folder.
- Keep the script template empty and create it.
- Confirm the script extends `Area2D`, which gives it `Area2D` behavior and functions.

3 Connect the body-entered signal ▶ 3:23

- Open the Signals tab in the **Inspector**.
- Find and connect the body_entered signal.
- This signal will detect when the player or another body enters the restart zone.
- Remove the default pass statement so you can add real restart logic.

4 Add a timer to create a smooth restart delay

▶ 4:38

- Add a `Timer` node as a child of the Restart Zone.
- Set the timer wait time to 0.5 seconds.
- Enable One Shot so the timer runs only once and does not loop.
- This delay creates a smoother restart effect instead of instantly reloading the scene.

5 Expose the timer as a variable in the script

▶ 5:33

- Drag the `Timer` node into the script and create a variable reference for it.
- Use the variable to control the timer from code.
- This allows the restart sequence to start automatically when the player enters the zone.

6 Start the timer when the player enters the zone

▶ 6:11

- In the body_entered function, call timer.start().
- This begins the 0.5-second countdown as soon as the player enters the restart zone.
- Use this for both fall-off-the-map and enemy-collision restart triggers.

7 Connect the timer timeout signal to reload the scene

▶ 6:42

- Connect the `Timer`'s timeout signal.
- In the timeout function, reload the current scene using get_tree().reload_current_scene().
- This restarts the level after the timer finishes.
- Make sure the function syntax is correct, including parentheses for get_tree().

8 Add the Restart Zone instance to the game level

▶ 8:08

- Open the main game level scene.
- Add an instance of the Restart Zone scene into the level.
- Since the base scene has no visible sprite or shape, it will not appear until you add a collision shape.

9 Add a collision shape specific to the level

▶ 9:11

- Select the Restart Zone instance in the level.
- Add a `CollisionShape2D` child node.
- Choose the collision shape that matches the use case, such as the world boundary shape for falling off the map.
- Adjust the z-index or ordering if needed so you can see and position it properly in the editor.
- Place it just below the ground or in the intended trigger location.

10 Test the restart behavior in-game

▶ 10:08

- Save the scene and run the game.
- Trigger the restart zone by falling into it or colliding with it.
- Confirm that the timer starts and the current scene reloads after the delay.
- Verify that the restart works repeatedly and consistently.

11 Optional: Slow down time for a dramatic restart effect

▶ 11:28

- In the Restart Zone script, set Engine.time_scale = 0.5 when the player enters the zone.
- This slows the game to half speed and gives a visual cue that the game is about to restart.
- When the timer times out, restore normal speed with Engine.time_scale = 1.
- This effect is optional but can improve the feel of the restart sequence.

⚠ **Watch out**

- Do not forget to reset Engine.time_scale to 1 after the restart sequence, or the game may remain slowed down.
- Ensure the timer is set to One Shot so it does not repeatedly trigger reloads.
- Use the correct scene reload command: get_tree().reload_current_scene().
- Add the collision shape only in the level instance if you want the Restart Zone scene to remain reusable across different trigger types.
- Double-check signal connections if the restart does not trigger; the body_entered and timeout signals must both be connected properly.

⚡ **Go faster**

- Build the Restart Zone as a reusable blueprint scene so you can drop it into multiple levels.
- Keep the base Restart Zone scene generic and add collision shapes only where needed.

- Use a short timer delay, such as 0.5 seconds, to keep the restart responsive while still feeling polished.
- If you plan to use multiple restart triggers, reuse the same script and only change the collision shape per level.
- Test after each major step: signal connection, timer start, scene reload, and optional slow-motion effect.

16 DAY 3 Create a Collectible Coin Object with Animation, Collision, and Player-Only Detection

This SOP explains how to build a reusable coin object for a game level, animate it, place multiple instances in the scene, and configure collision so only the player can collect it. It also covers organizing the coin into a parent node and adding the basic collection behavior in script.

🎥 Watch the whole video

1 Plan the coin as a reusable game object ▶ 0:34

- Treat the coin as a reusable object so it can be placed multiple times in the level.
- Create a separate blueprint/scene for the coin instead of building each coin manually in the level.
- Use instances of the coin scene whenever you need another coin in the game.
- Keep reusable objects organized so they can be updated once and reused everywhere.

2 Create the coin scene and required nodes ▶ 1:47

- Add a new scene and choose `Area2D` as the root node.
- Rename the root node to Coin.
- Add an `AnimatedSprite2D` node so the coin can spin or animate.
- Add a `CollisionShape2D` node so the coin can detect overlaps with the player.
- Confirm the node structure is complete before moving on.

3 Set up the coin animation

▶ 2:44

- Select `AnimatedSprite2D` and create a new `SpriteFrames` resource.
- Load the coin sprite sheet from the assets folder.
- Split the sprite sheet into individual frames.
  - Set the vertical frame count to 1.
  - Set the horizontal frame count to 12.
- Add all 12 frames to the animation.
- Set the animation frame rate to 10.
- Enable Autoplay so the animation starts automatically when the scene loads.
- Preview the animation to confirm the coin spins correctly.

4 Configure the collision shape and save the coin scene

▶ 4:25

- Select the `CollisionShape2D` node.
- Set the collision shape to a circle to match the coin's shape.
- Resize and position the circle so it fits closely around the coin sprite.
- Make sure the animated sprite is on the correct frame and visually aligned.
- Save the scene in the **Scenes folder** as coin.

5 Place coin instances in the game level

▶ 5:00

- Return to the main game level scene.
- Drag the coin scene into the level to create instances.
- Place coins in locations that encourage the player to move, jump, and navigate the platform.
- Add as many coins as needed for the level design.
- Verify the coins appear in the correct positions and are visible in the scene.

6 Fix coin visibility and organize coins under a parent node

▶ 6:10

- If coins do not appear correctly, check the Z index so the coin renders above the background.
- Make sure the correct node is highlighted before adding instances.
- If coins were accidentally added as children of another object, delete them and re-add them under the correct parent.
- Create a regular parent node named Coins under the game level.
- Drag all coin instances under the Coins node to keep the scene organized.

7 Add a script to the coin for collection behavior

▶ 8:01

- Open the coin scene and attach a script to the Coin root node.
- Save the script in the **Scripts folder** for better organization.
- Use the coin script to control what happens when the player touches the coin.
- Connect the body_entered signal so the coin can detect when something enters its area.
- Create the signal callback function to handle collection logic.

8 Make the coin disappear when collected

▶ 9:40

- In the signal callback, detect when the player enters the coin's collision area.
- Use queue_free() to remove the coin from the scene after collection.
- Set visible = false immediately when the coin is touched so it disappears right away.
- This helps the collection feel instant and avoids awkward delays later when sound effects are added.

9 Restrict coin collection to the player only

▶ 11:49

- Review the collision layers and masks so the coin only reacts to the player.
- Set the player to live in collision layer 2.
- Keep the coin on layer 1, but set its mask to detect layer 2 only.
- This ensures enemies or other objects do not accidentally collect the coin.
- Use collision layers and masks to control which objects can interact with each other.

10 Test coin collection in the game

▶ 15:10

- Save all scenes before testing.
- **Run the game** and move the player into the coins.
- Confirm the coin disappears when the player touches it.
- Verify that the player can collect multiple coins throughout the level.
- If the coin is collected successfully, the core mechanic is working as intended.

⚠ Watch out

- Make sure you are adding coin instances under the correct parent node; otherwise, they may become children of the wrong object.
- If the coin is not visible, check the Z index and confirm it renders above the background.
- Ensure the collision shape matches the coin's visual size so collection feels accurate.

- Be careful when using body_entered; without layer/mask filtering, non-player objects may trigger collection.
- If you later add sound effects, revisit the collection timing so the sound and coin removal happen cleanly together.

Go faster

- Build the coin as a reusable scene once, then instance it wherever needed.
- Keep related objects grouped under parent nodes like Coins for easier scene management.
- Save scenes and scripts frequently to avoid losing work.
- Use collision layers and masks instead of extra code when you only want one object type to interact.
- Preview the animation and test collection early so issues are caught before adding more gameplay systems.

17 DAY 3 Create a Patrolling Enemy with Boundary Detection and Restart Collision in a 2D Game

Build a simple enemy that animates, patrols between two invisible boundaries, and triggers a game restart when the player collides with it. This SOP also covers the required scene setup, collision layers, raycast configuration, and script logic needed for reliable behavior.

Watch the whole video

1 Create the enemy scene root and save it ▶ 0:39

- Create a new scene for the enemy.
- Add a `Node2D` as the root node.
- Rename the root node to Enemy.
- Save the scene in the **Scenes folder** as enemy.
- Keep the enemy scene simple so additional behavior can be added later.

2 Add and configure the enemy animation

▶ 1:34

- Select the Enemy root node.
- Add an `AnimatedSprite2D` child node.
- Create a new `SpriteFrames` resource.
- Use Add Sprite Sheet to import the slime enemy asset.
- Set the sprite sheet grid correctly (example shown: 3 columns).
- Choose the idle animation row for the enemy.
- Set the animation frame rate to 10.
- Enable Autoplay so the animation starts when the scene loads.
- Adjust the sprite position so the enemy is visible and centered properly.

3 Add a restart collision zone to the enemy

▶ 3:21

- Reuse the previously created Restart Zone scene.
- Attach the restart zone as a child of the enemy scene using the scene-link/instance option.
- Add a `CollisionShape2D` to the restart zone if needed.
- Set the collision shape to a rectangle for better fit around the enemy.
- Resize and position the shape so it covers the enemy body accurately.
- Confirm the restart zone will trigger a game restart when the player collides with the enemy.

4 Create a reusable platform boundary scene

▶ 8:08

- Create a new scene using an `Area2D` root node.
- Rename it to Platform Boundary.
- Add a `CollisionShape2D` child node.
- Set the collision shape to a rectangle.
- Make the rectangle thin and wide enough to act as an invisible edge detector.
- Save the scene in the **Scenes folder** as platform boundary.
- Use this scene repeatedly wherever enemy patrol limits are needed.

5 Place the enemy and boundary markers in the level

▶ 9:31

- Open the main game level scene.
- Drag the Enemy scene into the level and place it where the patrol should begin.
- Increase the enemy's Z index if it disappears behind other objects.
- Place one Platform Boundary instance on each side of the patrol area.
- Position the boundaries at the left and right edges of the enemy's allowed movement range.
- Ensure the boundaries also have a higher Z index if needed for visibility during editing.
- Use as many enemy/boundary sets as needed for the level layout.

6 Add movement logic to the enemy script

▶ 10:48

- Open the Enemy scene.
- Attach a new script to the enemy root node and save it in the **Scripts folder**.
- Keep the default _ready() and _process(delta) functions available.
- Remove or ignore _ready() if it is not needed.
- Create a constant named SPEED and set it to 60.
- Create a variable named direction and initialize it to 1.
- In _process(delta), update position.x every frame using:
  - position.x += direction SPEED delta
- Use delta to keep movement consistent across different frame rates.
- Add a code comment to document that this line makes the enemy move.

7 Add raycasts to detect patrol boundaries

▶ 15:22

- Under the Enemy node, add a RayCast2D child for the right side.
- Rename it to RaycastRight.
- Position it at the center of the enemy and rotate it to face right.
- Add a second RayCast2D child for the left side.
- Rename it to RaycastLeft.
- Position it at the center of the enemy and rotate it to face left.
- Configure both raycasts to detect only the boundary layer used by the platform boundaries.
- Set the raycast collision mask to Layer 3 for both raycasts.
- Move the Platform Boundary scene's collision layer to Layer 3 so the raycasts can detect it.

8 Store raycasts in script variables and reverse direction on collision

▶ 19:00

- In the enemy script, drag RaycastRight into the script to create a node reference variable.
- Drag RaycastLeft into the script to create a second node reference variable.
- In _process(delta), check whether RaycastRight.is_colliding().
- If the right raycast collides:
  - Set direction = -1
  - Set `AnimatedSprite2D`.flip_h = true so the enemy faces left
- Check whether RaycastLeft.is_colliding().
- If the left raycast collides:
  - Set direction = 1
  - Set `AnimatedSprite2D`.flip_h = false so the enemy faces right
- Keep the movement line active so the enemy continues moving based on the updated direction.

9 Fix raycast detection settings if the enemy does not move correctly

▶ 24:09

- If the enemy does not appear or patrol correctly, inspect the raycast settings.
- Open each raycast in the enemy scene.
- In the Collide With settings, ensure Areas are enabled.
- This is required because the platform boundaries are `Area2D` nodes.
- Save all scenes after updating the raycast settings.
- Re-test the level to confirm the enemy now detects the boundaries properly.

10 Test enemy patrol and restart behavior in the level

▶ 25:19

- **Run the game** and verify the enemy moves back and forth between the two boundaries.
- Confirm the enemy flips direction when it reaches each boundary.
- Move the player into the enemy to confirm the restart behavior triggers.
- Verify the game restarts as expected after collision.
- If the collision response looks abrupt, plan a later improvement such as a fall or death animation for the player.

⚠ Watch out

- Do not forget to save each scene after creating or modifying it.
- RayCast2D must be configured to collide with Areas if the boundaries are `Area2D` nodes.
- Collision layers and masks must match: boundaries on Layer 3, raycasts looking for Layer 3.

- If the enemy is hidden behind other objects, adjust its Z index.
- Use a rectangle collision shape for the enemy's restart zone if a circle does not fit well.
- The restart zone should only trigger on intended player contact; verify it does not overlap unrelated objects.

⚡ Go faster

- Reuse the Restart Zone and Platform Boundary scenes instead of rebuilding collision logic each time.
- Keep the enemy scene modular: animation, restart collision, and patrol detection should remain separate child nodes.
- Use a single patrol script pattern for multiple enemies by changing only placement and boundary positions.
- Start with one enemy and one boundary pair to validate the setup before duplicating it across the level.
- If a collision shape does not fit well, switch shapes early rather than trying to force a poor fit.

18 DAY 3 Add a Simple Player Fall Effect on Enemy Collision

Create a basic visual death effect by removing the player's `CollisionShape2D` when the player collides with an enemy or restart zone. This makes the character fall immediately, providing a simple and effective visual cue before the scene reloads.

🎥 Watch the whole video

1 Open the Restart Zone script ▶ 0:19

- In the project, locate and open the Restart Zone script.
- This script is where collision handling for the player/enemy interaction will be updated.
- Confirm you are editing the script that runs when the player enters the restart zone or collides with the enemy.

2 Capture the colliding body ▶ 0:26

- Use the collision callback's body parameter to reference whatever object entered the zone.
- Treat body as the player object when the player collides with the enemy.
- This allows you to access nodes inside the player from the script.

3 Access the player's `CollisionShape2D` node

▶ 1:14

- From the body reference, call getNode() to find the player's collision node.
- Target the player's `CollisionShape2D` node specifically.
- This node is what keeps the player physically colliding with the ground and other objects.

4 Remove the `CollisionShape2D` node to trigger the fall

▶ 1:25

- Delete the player's `CollisionShape2D` node using queue_free().
- Once the collision shape is removed, the player will no longer collide with the ground.
- The character will fall, creating a simple death effect.

5 Save and test the behavior

▶ 2:05

- Save the script changes.
- **Run the game** and collide with the enemy to confirm the player falls as expected.
- Verify that the scene reload still works after the fall effect is triggered.

6 Consider a more advanced death animation later

▶ 2:24

- If a more polished effect is needed, add a death animation to the player's `AnimatedSprite2D` node.
- A full death animation would also require a dedicated die/death function in the player script.
- This approach is more involved, so the collision-shape removal method is a simpler option for now.

⚠ Watch out

- Do not delete the entire player node unless that is the intended behavior; only remove the `CollisionShape2D` for the fall effect.
- Make sure the correct node name is used when calling getNode() so the script can find the collision shape.
- Test after saving to confirm the scene reload and fall behavior both work correctly.
- If the player does not fall, verify that the collision shape is actually attached to the player and that the script is running on the correct collision event.

⚡ Go faster

- Use the collision-shape removal method for a fast, lightweight visual effect.

- Keep the implementation simple first, then upgrade to a full death animation later if needed.
- Test immediately after each change to catch node-path or naming issues early.
- Reuse the existing restart/collision logic instead of building a separate death system from scratch.

19 DAY 3 Add On-Screen Instructions and Organize Scene Elements in a 2D Game

Create and position instructional text in the game UI, organize scene objects for easier management, and verify collision settings so the game behaves correctly. This SOP also covers basic visual cleanup and preparation for future polish items like sound effects and level-complete messaging.

🎥 Watch the whole video

1 Clean up the background so the camera view looks polished ▶ 0:30

- Open the background or environment layer in the scene.
- Fill in any visible empty/gray areas near the camera's travel path.
- Make sure the background remains visually consistent when the camera moves across the level.
- Recheck the far edge of the level and add color where needed so no unfinished areas appear during gameplay.
- Use the separation between foreground and background to make quick visual adjustments without affecting gameplay objects.

2 Group related objects into organized parent nodes ▶ 1:38

- Create a parent node for all enemy objects and rename it Enemies.
- Drag each enemy into the Enemies group so they are managed together.
- Create another parent node for boundary objects and rename it PlatformBoundaries.
- Move platform boundary objects into that group.
- Keep the scene hierarchy organized so it is easier to edit, troubleshoot, and expand later.
- Confirm the scene tree clearly shows major groups such as player, setting, platforms, restart zones, coins, enemies, and boundaries.

3 Verify restart-zone collision settings for the world boundary

▶ 2:39

- Locate the World Boundary restart zone in the scene.
- Confirm it is clearly renamed so its purpose is obvious.
- Open its collision settings.
- Set the collision mask to detect the player layer, specifically Layer 2.
- Verify the restart zone will trigger when the player falls off the level.
- Test the setup to ensure the game restarts correctly when the player collides with the world boundary.

4 Add a text label for player instructions

▶ 3:32

- Select the Game `Label` or UI layer where text should appear.
- Click the plus button and add a `Label` node.
- Position the label inside the camera view so players can see it during gameplay.
- Rename the label to Instructions `Label` for clarity.
- Enter instructional text such as:
  - Collect all coins
  - Avoid slime monsters
  - Reach the end of the level
- Keep the wording short and easy to read.

5 Adjust label layering, font, and size for readability

▶ 4:32

- If the label is not visible, adjust its Z-index / Z-frame so it renders above the background and other objects.
- Set the label to a layer that keeps it visible without completely covering gameplay elements.
- Move the label slightly if needed so it sits comfortably within the camera view.
- Open Theme and Theme Overrides to customize the font.
- Import a font from the asset folder if desired, such as a pixel-style font.
- Reduce the font size if the text appears too large; for example, lower it to around 8.
- Confirm the text is readable and visually matches the game style.

6 Playtest the game to confirm the instructions display correctly

▶ 5:51

- **Run the game** in playtest mode.
- Verify the instruction label appears on screen during gameplay.
- Check that the text is positioned clearly and does not block important gameplay elements.
- Confirm the player can still see the level and understand the objective.
- If the label is hard to read or poorly placed, return to the label settings and adjust font size, position, or layering.

7 Validate gameplay behavior and identify remaining polish items

▶ 6:44

- Add optional control instructions if needed, such as:
  - Space = Jump
  - A = Move Left
  - D = Move Right
- Test collision outcomes:
  - Falling off the level should restart the game.
  - Hitting an enemy should restart the game.
- Confirm the game is functional and the player can complete the basic loop.
- Note remaining polish tasks to finish later:
  - Jump sound effect
  - Coin pickup sound effect
  - End-of-level completion message

⚠ Watch out

- Do not forget collision masks on restart zones; incorrect layer settings will prevent restart behavior from working.
- Keep UI text inside the camera view so it remains visible during gameplay.
- Avoid oversized fonts that can cover the play area or become difficult to read.
- Check Z-order/layering if text appears behind other objects.
- Test after each change to catch visibility or collision issues early.

⚡ Go faster

- Group related objects early to reduce scene clutter and make future edits faster.
- Use short, direct instruction text so players can understand the objective immediately.
- Reuse a consistent font style across labels to keep the UI cohesive.
- Make small background fixes before adding UI so the final scene looks polished.

- After adding a label, always run a quick playtest to confirm placement, readability, and layering.

20 DAY 3 Add and Configure Jump and Coin Sound Effects in a Game Project

This SOP explains how to add sound effects for player jumping and coin collection, route them through separate audio buses, and ensure each sound plays correctly before the related object is removed. Follow these steps to keep audio organized and prevent sounds from cutting off.

Watch the whole video

1 Create dedicated audio buses for sound effects ▶ 0:18

- Open the Audio panel at the bottom of the editor.
- Review the existing Master Bus, which controls all project audio.
- Create a new bus named SFX to manage sound effects separately.
- Optionally create additional buses for individual sounds, such as:
  - Jump SFX
  - Coin SFX
- Use separate buses when you want independent volume and mix control for each sound.

2 Add an `AudioStreamPlayer2D` to the player for jump sounds ▶ 1:37

- Open the Player scene and select the player node.
- Add a new child node of type `AudioStreamPlayer2D`.
- This node will be used to play the jump sound from the player character.
- In the **Inspector**, assign the jump audio file from the assets folder (for example, jump.wav) to the Stream property.
- Test the sound using the play button to confirm the file is loaded correctly.
- Set the node's bus to Jump SFX so the jump sound can be adjusted independently.

3 Reference the player audio node in the player script and play it on jump

▶ 2:53

- In the player script, drag the `AudioStreamPlayer2D` node into the script while holding Ctrl to create a variable reference.
- Locate the jump logic in the code where the player is allowed to jump.
- Immediately after the jump velocity is applied, call:
  - audio_stream_player.play()
- This ensures the jump sound plays every time the player jumps.
- Save the scene after confirming the reference is correctly connected.

4 Add an `AudioStreamPlayer2D` to the coin for collection sounds

▶ 3:56

- Open the Coin scene and select the coin node.
- Add a new child node of type `AudioStreamPlayer2D`.
- Assign the coin sound file from the assets folder (for example, coin.wav) to the Stream property.
- Use the play button to verify the sound is loaded and audible.
- Set the node's bus to Coin SFX so the coin sound can be mixed separately from other audio.

5 Reference the coin audio node and play it before removing the coin

▶ 4:23

- In the coin script, drag the `AudioStreamPlayer2D` node into the script while holding Ctrl to create a variable reference.
- In the coin collection logic, play the sound before deleting the coin.
- Call the play function on the audio player immediately before the coin is hidden or removed.
- This ensures the collection sound starts as soon as the player picks up the coin.

6 Prevent the coin sound from cutting off before the coin is destroyed

▶ 5:11

- Recognize that calling queue_free() too early will destroy the coin and its child nodes, including the audio player.
- To avoid cutting off the sound, first make the coin invisible.
- Use an await statement to wait for the audio to finish playing before deleting the coin.
- Example flow:
  - Play the coin sound
  - Set the coin visibility to false
  - await the audio player's finished signal
  - Then call queue_free()
- This prevents the sound from being interrupted and avoids a visible delay in the coin's removal.

7 Save and test the updated audio setup

▶ 7:16

- Save all scenes after adding both sound effects.
- **Run the game** and test both actions:
  - Jumping should trigger the jump sound.
  - Collecting a coin should trigger the coin sound.
- Confirm that each sound plays through its assigned bus.
- Adjust bus levels if one sound is too loud or too quiet.
- Verify that the coin disappears cleanly after the sound finishes.

⚠ Watch out

- Do not call queue_free() on the coin before the sound has finished, or the audio may cut off.
- If the audio node is a child of the coin, destroying the coin will also destroy the audio player.
- Make sure the correct bus is assigned to each sound so you can control them independently.
- If the sound is not audible during testing, check your project audio settings and output volume.

⚡ Go faster

- Use separate buses for each major sound type to make balancing easier later.
- Reuse the same workflow for other game sounds, such as damage, power-ups, or menu clicks.
- Test each sound immediately after adding it to confirm the file, bus, and script reference are correct.

- Keep sound playback calls close to the gameplay event they belong to for easier maintenance.

21 DAY 3 Build Score Tracking and End-of-Level Win Message in a 2D Game

Create a game manager to track coin-based score, display the score on-screen, and trigger a win message when the player reaches the end-of-level area. This SOP also includes setup, scripting, testing, and debugging steps to ensure the system works reliably.

Watch the whole video

1 Add a Game Manager to Centralize Score Logic ▶ 0:48

- Open the main game level scene.
- Add a new regular node to the scene.
- Rename the node to GameManager.
- Position it near the top of the scene tree for easy access.
- Use a standalone node for systems that should exist once per level, such as score tracking.

2 Attach a Script and Create the Score Variable ▶ 2:09

- Select GameManager.
- Attach a new script and save it in the **Scripts folder**.
- Create a score variable in the script:
  - `var score = 0`
- Treat score as the number of coins collected.
- Keep the script lightweight because the node is only used for game logic, not visuals.

3 Make GameManager Accessible from Other Nodes ▶ 4:12

- Right-click GameManager in the scene tree.
- Enable Access as Unique Name.
- This allows other nodes, such as coins, to reference GameManager safely.
- Avoid using a hard-coded file path reference, because moving files can break the connection.
- Confirm the node shows the unique-name indicator before continuing.

4 Create a Function to Increase Score

▶ 6:24

- In the GameManager script, add a function named addScore.
- Inside the function, increase the score by one:
  - score += 1
- This function will be called whenever the player collects a coin.
- Keep the function simple so it can be reused consistently.

5 Call GameManager from the Coin Script

▶ 7:12

- Open the coin script.
- Reference GameManager using its unique name.
- When the coin is collected, call:
  - GameManager.addScore()
- This ensures the score increases every time the player touches a coin.
- Test the reference carefully to confirm the coin can communicate with the manager.

6 Add a Score `Label` to Display the Current Score

▶ 7:46

- Select GameManager.
- Add a `Label` as a child node.
- Rename it to ScoreLabel.
- Move the label to a visible area of the screen.
- Set the initial text to something like:
  - Coins collected: 0
- Adjust font, size, and wrapping so the label is readable in-game.

7 Reference the Score `Label` in the GameManager Script

▶ 10:06

- In the GameManager script, drag ScoreLabel into the script while holding Ctrl.
- Update the label text whenever score changes.
- Set the label text using concatenation:
  - ScoreLabel.text = "Coins collected: " + str(score)
- Use str(score) to convert the number into text.
- Make sure spacing and punctuation are correct so the label reads cleanly.

8 Save and Verify Score Updates

▶ 12:36

- Confirm the coin script calls GameManager.addScore().
- Confirm the GameManager script updates ScoreLabel.text.
- Save all scenes before testing.
- **Run the game** and collect coins to verify the score increases on-screen.
- If the score does not display correctly, recheck the unique-name reference and label text formatting.

9 Create an End-of-Level Trigger Scene

▶ 13:16

- Create a new scene for the end-of-level trigger.
- Use an `Area2D` node as the root.
- Rename it to EndOfGame.
- Add a `Sprite2D` child to represent the trigger visually.
- Use a simple object such as an apple or door graphic to indicate the finish point.

10 Add Collision and Configure the Trigger Mask

▶ 15:29

- Add a `CollisionShape2D` to the EndOfGame `Area2D`.
- Size the collision shape so it surrounds the trigger sprite.
- Set the collision mask so it detects only the player.
- Match the player's collision layer/mask setup to avoid false triggers from enemies or other objects.
- Save the end-of-game scene after setup.

11 Place the End-of-Game Trigger in the Main Level

▶ 16:49

- Return to the main game level scene.
- Drag the EndOfGame scene into the level near the finish area.
- Adjust its draw order if it appears behind other objects.
- Scale the trigger up if needed so it is easy to reach and see.
- Position it where the player should end the level.

12 Add a Hidden Win Message to the Trigger Scene

▶ 18:12

- Open the EndOfGame scene.
- Add a `Label` as a child of the EndOfGame node.
- Rename it to CongratsMessage.
- Set the label text to something like:
  - Congrats, you win
- Adjust font size and wrapping so the message is readable.
- Set the label's visibility to false so it does not appear at the start.

13 Attach a Script and Connect the Body-Entered Signal

▶ 19:46

- Attach a script to the EndOfGame node.
- Save the script in the **Scripts folder**.
- Drag CongratsMessage into the script while holding Ctrl.
- Connect the body_entered signal for the `Area2D`.
- Ensure the signal is connected in the correct script, not in GameManager.

14 Show the Win Message When the Player Reaches the End

▶ 21:07

- In the EndOfGame script, handle the body_entered signal.
- When the player enters the area, set:
  - CongratsMessage.visible = true
- This makes the win message appear only when the player reaches the end trigger.
- Confirm the collision mask is correct if the signal does not fire.

15 Test the Full Game Flow and Debug as Needed

▶ 22:25

- **Run the game** and verify coin collection updates the score.
- Reach the end-of-level trigger and confirm the win message appears.
- If the message does not appear, check:
  - Whether the label is a child of the correct scene
  - Whether the script references are correct
  - Whether the collision mask is set properly
- If needed, delete and re-add the trigger or label to fix scene-tree issues.
- Retest after each fix until the score and win message both work correctly.

⚠ Watch out

- Do not use a hard-coded node/file path for GameManager if you can avoid it; moving files can break references.

- Make sure ScoreLabel and CongratsMessage are children of the nodes that reference them in script, or the drag-and-drop unique reference may not work as expected.
- Be careful not to place code in the wrong script; the transcript shows a debugging issue caused by editing the wrong node's script.
- Verify collision masks and layers carefully so the end trigger only responds to the player.
- If a label is hard to read, increase font size and adjust wrapping rather than leaving it too small.

⚡ Go faster

- Use unique names for singleton-style nodes like GameManager to simplify cross-node communication.
- Keep score logic in one place so coins only need to call a single function.
- Test after each major change: score update, label display, trigger detection, and win message visibility.
- Use child nodes for UI elements that belong to a system node; this makes referencing them easier and safer.
- If a scene element looks wrong, try deleting and re-adding it rather than spending too long patching a broken setup.

Polished from the BGC "Leading Through Code: Owning Your Superpower" playbook appendix — same steps, same Loom links, camp colour-coding. 21 SOPs | Day1 6 · Day2 8 · Day3 7.